GeoClaw-claude

Python GIS Analysis Toolkit

Technical Reference

Version 2.2.1 | March 2025

UrbanComp Lab

https://urbancomp.net

MIT License | Copyright © 2025 UrbanComp Lab

1. System Architecture

1.1 Module Overview

GeoClaw-claude is organized into the following top-level modules under geoclaw_claude/:

Module	Description
core/	GeoLayer (core vector layer class) and GeoClawProject (project management, layer registry, metadata)
io/	vector.py (GeoJSON/SHP/GML read-write), osm.py (Overpass/OSM download), remote.py (HTTP/WFS/Tianditu)
analysis/spatial_ops.py	buffer(), clip(), nearest_neighbor(), kde(), zonal_stats() — all CRS-aware with UTM reprojection
analysis/network.py	build_network(), isochrone(), shortest_path(), batch_service_area() — OSMnx-based
analysis/raster_ops.py	load_raster(), slope(), aspect(), hillshade(), reclassify(), raster_calc(), zonal_stats()
analysis/mobility/	Human mobility: read_positionfixes(), generate_full_hierarchy(), metrics, visualization (trackintel)
nl/	NLProcessor (intent parsing), NLExecutor (GIS execution), GeoAgent (multi-turn dialog)
memory/	ShortTermMemory, LongTermMemory, MemoryManager — cross-session knowledge accumulation
cartography/	renderer.py (4 map themes + Folium), map_composer.py (multi-layer layout, scale bar, legend)
utils/coord_transform.py	WGS84 ↔ GCJ-02 ↔ BD-09 point-level and layer-level transform
skills/	SkillManager, SkillContext, builtin skills (hospital_coverage, etc.)
updater.py	check(), update(), self_check() — version detection, git pull, health report
config.py	JSON-based config + env variable override (geoclaw_claude_*), onboard wizard
cli.py	Click CLI entry point: ask/chat/memory/check/update/self-check/skill/download

1.2 Dependency Graph (Simplified)

The module dependency is intentionally kept minimal to reduce coupling:

```
cli.py
  ├─ nl/agent.py  →  nl/executor.py  →  analysis/*  →  core/layer.py
  │                →  memory/manager.py
  ├─ updater.py
  ├─ memory/manager.py  →  short_term.py  +  long_term.py
  ├─ skill_manager.py  →  skills/
  └─ config.py

analysis/mobility/core.py  →  trackintel (external)
analysis/network.py        →  osmnx / networkx
analysis/raster_ops.py     →  rasterio / numpy
io/osm.py                  →  osmnx / overpy
utils/coord_transform.py   →  pure math (no external deps)
```

2. Natural Language Module (nl/)

The NL module implements a two-stage pipeline: intent parsing (NLProcessor) and GIS execution (NLExecutor). GeoAgent wraps both into a conversational multi-turn interface with Memory integration.

2.1 NLProcessor

Intent Parsing Pipeline

NLProcessor.parse(text) returns a ParsedIntent dataclass:

```
@dataclass
class ParsedIntent:
    action:      str            # e.g. 'buffe', 'mobility_hierarchy'
    params:      dict           # extracted parameters
    targets:     list[str]      # target layer names
    confidence:  float          # 0.0 ~ 1.0
    explanation: str            # human-readable description
    steps:       list[dict]     # multi-step pipeline decomposition
```

Dual-Mode Operation

Mode	Description
AI Mode	Calls the Claude API (claude-3-haiku) with a structured system prompt. Handles ambiguous, compound, and domain-specific commands. Activated automatically when anthropic_api_key is set.
Rule Mode	Keyword lists + regex patterns. Covers 30+ action types. Fully offline. Auto-activated when no API key is configured. Can be forced with --rule flag.

Action Registry

All recognized actions and their keyword triggers (rule mode):

Action	Trigger Keywords / Patterns
load	加载/读入/load/open/import + file extension
buffer	缓冲区/buffer/扩展范围 + distance pattern (e.g. 1km, 500m)
clip	裁剪/clip/mask/extract by boundary
nearest_neighbor	最近邻/nearest/closest + neighbor/station
kde	核密度/kde/density/热力图 (before mobility_heatmap check)
zonal_stats	分区统计/zonal/statistics by zone/district
isochrone	等时圈/isochrone/reachability + minutes pattern

Action	Trigger Keywords / Patterns
shortest_path	最短路径/shortest/path/route
coord_transform	坐标转换/transform/convert + crs name
download_osm	下载/download + osm/data type
render	可视化/visualize/map/plot (before mobility_plot check)
mobility_load	轨迹/gps/positionfix/mobility data
mobility_staypoints	停留点/staypoint/staypoint detection
mobility_triplegs	出行段/tripleg/出行模式
mobility_transport	预测出行方式/出行方式预测/predict transport mode
mobility_hierarchy	移动性分析/一键完成/full hierarchy
mobility_locations	重要地点/location/识别家和工作地
mobility_summary	移动性指标/mobility metrics/回转半径
mobility_plot	轨迹地图/trajectory map (before render check)
mobility_heatmap	时间热力图/activity heatmap (before kde check)
mobility_modal	出行方式构成图/modal split/出行方式构成

2.2 NLExecutor

NLExecutor.execute_intent(intent, context) maps ParsedIntent to actual function calls. It maintains a named layer dict (context) that persists across turns in GeoAgent.

Parameter Extraction

Numeric parameters are extracted from natural language using regex patterns:

Parameter Type	Example → Extracted Value
Distance	"1km" / "500m" / "1.5 公里" → meters (float)
Minutes	"10 分钟" / "15 min" → minutes (int)
Grid size	"100x100" / "grid 50" → intege
Bandwidth	"bandwidth 0.05" / "bw=0.1" → float

Parameter Type	Example → Extracted Value
Threshold distance	"80m threshold" / "dist_threshold=100" → meters
Threshold time	"5min threshold" / "time_threshold=10" → minutes

Layer Context Management

Executor maintains a layer context dict. Layers are named automatically (buffer_result, staypoints, triplegs, etc.) or by user-specified names. Previous operations' output is automatically passed as input to subsequent operations in a pipeline.

2.3 GeoAgent

GeoAgent wraps NLProcessor and NLExecutor into a conversational interface. Key behaviors:

- Layer context persists across all turns in the same session
- All executed operations are logged to ShortTermMemory
- Calls mem.end_session() on agent.end(), flushing session summary to LongTermMemory
- Confidence threshold (default 0.6): below this, asks user for confirmation before executing
- Multi-step pipelines (e.g. 'load then buffer then visualize') are decomposed and executed sequentially

3. Mobility Analysis Module (analysis/mobility/)

This module wraps the trackintel library to provide a consistent API for human mobility analysis. All public functions are re-exported from the __init__.py module. Algorithm implementations belong to trackintel; GeoClaw-claude provides the API layer and integration.

3.1 core.py — Data Hierarchy Generation

read_positionfixes()

```
def read_positionfixes(
    source,                    # str (CSV path) | GeoDataFrame | DataFrame
    *,
    user_id_col:  str = "user_id",
    tracked_at_col: str = "tracked_at",
    lon_col:      str = "longitude",
    lat_col:      str = "latitude",
    crs:          str = "EPSG:4326",
) -> GeoDataFrame:
```

Reads GPS positionfixes into trackintel-compatible format. Handles timezone normalization (UTC), coordinate parsing, and GeoDataFrame construction. Returns a trackintel Positionfixes GeoDataFrame.

generate_staypoints()

```
def generate_staypoints(
    positionfixes: GeoDataFrame,
    *,
    dist_threshold: float = 100,  # meters
    time_threshold: float = 5,    # minutes
    gap_threshold:  float = 30,   # minutes (data gap tolerance)
) -> tuple[GeoDataFrame, GeoDataFrame]:
```

Detects staypoints using a sliding window algorithm: a stay is detected when a user remains within dist_threshold meters for at least time_threshold minutes. Returns (updated_positionfixes, staypoints).

generate_triplegs()

```
def generate_triplegs(
    positionfixes: GeoDataFrame,
    staypoints:    GeoDataFrame,
    *,
    gap_threshold: float = 30,    # minutes
) -> tuple[GeoDataFrame, GeoDataFrame, GeoDataFrame]:
```

Extracts triplegs as continuous movement segments between staypoints. Returns (positionfixes, staypoints, triplegs).

generate_trips()

```
def generate_trips(
    staypoints: GeoDataFrame,
```

```
    triplegs:   GeoDataFrame,
) -> tuple[GeoDataFrame, GeoDataFrame, GeoDataFrame]:
```

Aggregates triplegs between activity staypoints into complete trips. Automatically adds is_activity=True to staypoints if not present (required by trackintel >= 1.3). Returns (staypoints, triplegs, trips).

generate_locations()

```
def generate_locations(
    staypoints: GeoDataFrame,
    *,
    epsilon:    float = 100,      # DBSCAN clustering radius (meters)
    num_samples: int  = 1,        # min staypoints per cluster
) -> tuple[GeoDataFrame, GeoDataFrame]:
```

Clusters staypoints using DBSCAN to identify semantically important locations (home, work, frequent venues). Returns (staypoints_with_location_id, locations).

generate_full_hierarchy() [convenience wrapper]

```
def generate_full_hierarchy(
    positionfixes: GeoDataFrame,
    *,
    dist_threshold:    float = 100,
    time_threshold:    float = 5,
    gap_threshold:     float = 30,
    location_epsilon:  float = 100,
    predict_mode:      bool  = False,
) -> dict:
```

One-call convenience function that runs the entire hierarchy generation pipeline (positionfixes → staypoints → triplegs → trips → locations) and optionally predicts transport modes. Returns a dict with keys: positionfixes, staypoints, triplegs, trips, locations.

predict_transport_mode()

```
def predict_transport_mode(
    triplegs: GeoDataFrame,
    *,
    method: str = "simple-coarse",  # 'simple-coarse' | 'random-forest'
) -> GeoDataFrame:
```

Adds a mode column to triplegs using speed-based heuristics (simple-coarse) or a random forest model (random-forest). Mode labels: slow_mobility, motorized_mobility, fast_mobility (simple-coarse) or walk/bike/car/train (random-forest).

3.2 metrics.py — Mobility Metrics

radius_of_gyration()

```
def radius_of_gyration(staypoints: GeoDataFrame) -> DataFrame:
```

Computes the radius of gyration per user — a measure of the spatial extent of mobility. Returns a DataFrame with columns user_id and radius_gyration (meters). Delegates to trackintel.analysis.radius_gyration().

jump_lengths()

```
def jump_lengths(staypoints: GeoDataFrame) -> DataFrame:
```

Computes the distribution of jump lengths (straight-line distances between consecutive staypoints) per user. Returns a DataFrame with columns user_id and jump_length (meters). Delegates to trackintel.analysis.jump_length().

modal_split()

```
def modal_split(
    triplegs: GeoDataFrame,
    *,
    metric: str = "count",  # 'count' | 'duration' | 'distance'
) -> dict:
```

Returns transport mode shares as a dict {mode: share} where shares sum to 1.0. Metric selects whether to weight by trip count, total duration, or total distance.

tracking_quality()

```
def tracking_quality(
    data: GeoDataFrame,         # positionfixes or staypoints
    *,
    granularity: str = "day",   # 'day' | 'hou'
) -> DataFrame:
```

Computes temporal tracking quality (fraction of time covered by data) per user per granularity period. For positionfixes, uses tracked_at; for staypoints, uses started_at/finished_at. Returns a DataFrame with user_id, day/hour, and quality columns.

identify_home_work()

```
def identify_home_work(
    staypoints: GeoDataFrame,
    locations:  GeoDataFrame,
    *,
    method: str = "osna",   # 'osna' | 'freq'
) -> GeoDataFrame:
```

Labels locations as home, work, or None using the OSNA method (activity patterns by time-of-day and day-of-week) or frequency method (most visited location). Returns locations GeoDataFrame with an added purpose column. Delegates to trackintel.analysis.location_identifier().

mobility_summary()

```
def mobility_summary(hierarchy: dict) -> dict:
```

Computes a comprehensive summary of all mobility metrics from a hierarchy dict. Returns a flat dict with keys:

Key	Value
n_users	Number of unique users

Key	Value
n_positionfixes	Total GPS point count
n_staypoints	Total staypoint count
n_triplegs	Total tripleg count
n_locations	Total location count
radius_of_gyration_m	Dict: {mean, median, max} in meters
jump_length_m	Dict: {mean, median, p90} in meters
modal_split	Dict: {mode: share} (count-based)
tracking_quality_mean	Mean daily tracking quality (0~1)

3.3 visualization.py — Mobility Visualization

All plots use a dark theme (background #1a1a2e) with English labels. All functions return a matplotlib Figure and optionally save to a file.

plot_mobility_layers()

```
def plot_mobility_layers(
    hierarchy: dict,
    *,
    user_id=None,              # filter to single user
    show_positionfixes=False,
    show_staypoints=True,
    show_triplegs=True,
    show_locations=True,
    title: str = "Mobility Data Layers Map",
    figsize=(12, 10),
    save_path=None,
) -> Figure:
```

Renders a layered dark-theme mobility map. Triplegs are colored by transport mode (slow_mobility: teal, motorized: red, fast: yellow). Staypoints are shown as orange dots. Locations as red markers with white edge.

plot_activity_heatmap()

```
def plot_activity_heatmap(
    staypoints: GeoDataFrame,
    *,
    user_id=None,
    title: str = "Activity Time Heatmap (Weekday x Hour)",
    figsize=(14, 5),
    save_path=None,
```

```
) -> Figure:
```

Renders a Weekday x Hour heatmap (7 rows × 24 columns) using YlOrRd colormap. Counts staypoint occurrences per weekday-hour combination.

plot_modal_split()

```
def plot_modal_split(
    triplegs: GeoDataFrame,
    *,
    metric='count',            # 'count' | 'duration'
    title: str = "Modal Split",
    figsize=(10, 5),
    save_path=None,
) -> Figure:
```

Renders a side-by-side pie chart and horizontal bar chart of transport mode shares.

plot_mobility_metrics()

```
def plot_mobility_metrics(
    summary: dict,
    *,
    title: str = "Mobility Metrics Dashboard",
    figsize=(12, 7),
    save_path=None,
) -> Figure:
```

Renders a multi-panel dashboard: dataset overview (stats card), radius of gyration bar chart, modal split pie, jump length bar chart, and tracking quality arc gauge.

4. Memory Module (memory/)

4.1 ShortTermMemory

Thread-safe in-memory storage (RLock). Auto-evicts entries after TTL expires.

Key Methods

Method	Description
start_session(session_id)	Initialize a new session with metadata timestamp
remember(key, value, ttl=3600)	Store value with TTL (seconds). Overwrites existing.
recall_short(key)	Retrieve value. Returns None if expired or missing.
log_op(op_type, description, **meta)	Append an operation to the session log (list)
set_context(key, value)	Set a session-scoped key-value pair
get_context(key)	Retrieve a context value
get_ops_log()	Return the full operation log for this session
clear()	Clear all in-memory storage for the current session

4.2 LongTermMemory

Persists entries to ~/.geoclaw_claude/memory/ as JSON files. Each entry is a separate JSON file named by entry ID and timestamp.

Entry Schema

```
{
  "id":         "uuid-string",
  "title":      "Analysis result title",
  "content":    { ... },          // arbitrary JSON
  "tags":       ["wuhan", ...],
  "importance": 0.8,              // 0.0 ~ 1.0
  "category":   "session",        // session | knowledge | result
  "created_at": "2025-03-01T...",
  "ops_log":    [ ... ],          // operation sequence (session entries)
}
```

Key Methods

Method	Description
learn(title, content, tags,	Store a new long-term memory entry

Method	Description
importance)	
recall(query, n=10)	Keyword search across title, tags, and content text
recall_recent(n=5)	Return n most recently created entries
recall_important(n=5, threshold=0.5)	Return n highest-importance entries above threshold
forget(entry_id)	Delete a specific entry by ID
compact(max_entries=1000)	Prune lowest-importance entries if count exceeds limit
export(path)	Export all entries to a single JSON file
search_by_tags(tags)	Return entries matching any of the given tags

4.3 MemoryManager

Singleton that combines ShortTermMemory and LongTermMemory. Accessed via get_memory(). Provides a unified interface and handles session lifecycle.

Key Methods

Method	Description
start_session(session_id)	Initialize short-term session
end_session(title, tags, importance)	Flush session (ops log + results) to long-term memory
remember/recall_short()	Delegates to ShortTermMemory
learn/recall()	Delegates to LongTermMemory
log_op/set_context/ get_context()	Delegates to ShortTermMemory
get_session_summary()	Dict with current session stats: ops count, duration, result count

5. Analysis Modules

5.1 spatial_ops.py

buffer(layer, distance, unit='meters')

Buffers all geometries by distance. Reprojects to UTM automatically for metric accuracy. Returns a GeoDataFrame in the original CRS with a geometry column of buffered polygons.

clip(layer, mask)

Clips layer to the mask geometry using geopandas overlay. CRS of layer is used; mask is reprojected if necessary.

nearest_neighbor(source, target, k=1)

Computes the k nearest features in target for each feature in source using a spatial KDTree (UTM-projected). Returns source GeoDataFrame with added distance_m and nearest_id columns.

kde(layer, bandwidth=0.01, grid_size=100)

Computes a Gaussian KDE density surface over the laye's point geometries. Returns a numpy array and a rasterio-compatible affine transform. Bandwidth is in CRS units (degrees for WGS84); use a projected CRS for metric bandwidth.

zonal_stats(raster, zones, stats=['mean','max','min','sum'])

Computes raster statistics for each zone polygon. Returns zones GeoDataFrame with added stat columns. Uses rasterstats internally.

5.2 network.py

build_network(bbox_or_place, network_type='drive')

Downloads OSM road network using osmnx. network_type: 'drive', 'walk', 'bike', 'all'. Returns a NetworkX MultiDiGraph.

shortest_path(G, origin, destination, weight='length')

Computes shortest path between two (lon, lat) coordinates using Dijkstra. Snaps points to nearest graph nodes. Returns a GeoDataFrame of the path geometry and a dict of path metrics (length_m, nodes).

isochrone(G, center, minutes=[5,10,15], network_type='walk')

Generates isochrone polygons (equal-travel-time contours) from a center point. Returns a GeoDataFrame with one polygon per time interval.

batch_service_area(G, facilities, minutes=[10])

Computes service areas for a set of facility points. Returns merged isochrone polygons per time interval.

5.3 raster_ops.py

Function	Description
load_raster(path)	Load GeoTIFF as (data_array, meta_dict)
slope(dem)	Compute slope in degrees from DEM array
aspect(dem)	Compute aspect (0-360 degrees) from DEM array
hillshade(dem, azimuth=315, altitude=45)	Compute hillshade for visualization
reclassify(raster, rules)	Apply reclassification rules: list of (min, max, new_value) tuples
raster_calc(raster_a, raster_b, expression)	Raster calculator: expression string e.g. 'a + b', 'a / (a + b)'
zonal_stats(raster, zones)	Same as spatial_ops.zonal_stats but takes (data, meta) tuple
clip_raster(raster, mask_geom)	Clip raster to polygon geometry (rasterio mask)
resample(raster, scale_factor)	Resample raster by scale factor

6. Core Layer & I/O

6.1 GeoLayer (core/layer.py)

GeoLayer wraps a GeoDataFrame with metadata, provenance tracking, and serialization support. It is the standard layer object passed between analysis functions.

Constructor

```
GeoLayer(
    data:  GeoDataFrame | str,     # GeoDataFrame or file path
    name:  str,
    crs:   str = None,             # auto-detected if None
    meta:  dict = None,            # user metadata
)
```

Key Attributes / Methods

Attribute / Method	Description
layer.gdf	Underlying GeoDataFrame
layer.name	Layer name (string identifier)
layer.crs	Coordinate reference system (e.g. 'EPSG:4326')
layer.meta	Arbitrary metadata dict
layer.reproject(crs)	Return new GeoLayer in target CRS
layer.to_file(path, driver='GeoJSON')	Save to file
layer.to_utm()	Reproject to appropriate UTM zone (auto-detected)
layer.bbox	Bounding box as (minx, miny, maxx, maxy)
layer.n_features	Number of features (int)
layer.geometry_type	Dominant geometry type string

6.2 I/O Modules

io/vector.py

Function	Description
load_vector(path)	Load GeoJSON/SHP/GPKG/GML to GeoLayer
save_vector(layer, path, driver)	Save GeoLayer to file

Function	Description
list_columns(layer)	Return column name/type dict

io/osm.py

Function	Description
download_osm(place_or_bbox, **tags)	Download OSM features via Overpass API. Returns GeoDataFrame.
download_osm_network(place, network_type)	Download road network (wraps osmnx.graph_from_place)

io/remote.py

Function	Description
fetch_wfs(url, layer_name)	Download a WFS layer to GeoDataFrame
fetch_http(url, file_type)	Download remote GeoJSON/SHP to GeoLayer
tianditu_poi(lon, lat, radius, keyword, api_key)	Query Tianditu POI API (China)

7. Updater & Configuration

7.1 updater.py

check()

```
def check() -> dict:
```

Fetches __init__.py from the main branch of the GitHub repository, extracts __version__, and compares with the local version. Returns a dict:

```
{ "local": "2.2.1", "remote": "2.2.1", "up_to_date": True,
  "latest_commit": "e9a87f1...", "message": "..." }
```

update(force=False)

```
def update(force: bool = False, run_tests: bool = False) -> dict:
```

Runs git pull origin main followed by pip install -e . in the package root. If run_tests=True, runs the test suite and reports pass/fail. Returns a dict with success, output, and test_results keys.

self_check()

```
def self_check(quick: bool = False) -> dict:
```

Performs a comprehensive health check. Checks: local version, remote version (unless quick=True), module imports (18 modules), dependency versions, config file validity, and data directory accessibility. Returns a structured dict with a pass/fail per component.

7.2 config.py

Configuration is stored as JSON at ~/.geoclaw_claude/config.json. All settings can also be set via environment variables prefixed with GEOCLAW_CLAUDE_ (uppercase). Environment variables take precedence over the config file.

Key Settings

Setting	Description
anthropic_api_key	Anthropic API Key. Required for NL AI mode and Skill AI calls.
data_dir	Default directory for data files. Default: ~/geoclaw_claude_data/
output_dir	Default output directory. Default: ~/geoclaw_claude_output/
cache_dir	Cache directory for OSM downloads etc. Default: ~/.geoclaw_claude/cache/
proxy	HTTP proxy URL (optional)
default_crs	Default output CRS. Default: EPSG:4326
nl_confidence_threshold	NL confidence threshold for auto-confirm. Default: 0.6

8. Testing & Development

8.1 Test Suite

File	Coverage
tests/test_memory.py	37 tests — ShortTermMemory, LongTermMemory, MemoryManager, session flush, concurrent access
tests/test_updater.py	20 tests — version parsing, check(), update() dry-run, self_check() components, changelog diff
tests/test_nl.py	20 tests — NLProcessor rule mode, AI mode mock, NLExecutor pipeline, GeoAgent multi-turn
tests/test_mobility.py	20 tests — read_positionfixes(), full hierarchy, metrics, visualization (save_path), version check

Run all tests:

```
python tests/test_memory.py
python tests/test_updater.py
python tests/test_nl.py
python tests/test_mobility.py
```

8.2 Development Conventions

- All public functions include docstrings with Parameters, Returns, and Example sections
- CRS handling: always reproject to UTM for metric distance calculations; return results in original CRS
- NL action keywords are defined as module-level constants (ALL_CAPS lists) in processor.py for easy extension
- New mobility operations: add keyword to processor.py constants, add handler to executor.py, add test to test_mobility.py
- Version number: update __version__ in geoclaw_claude/__init__.py + CHANGELOG.md + test files

8.3 Adding a New NL Operation

Step-by-step guide to adding a new natural language operation:

1. Define a keyword constant in processor.py: MY_OP = ["keyword1", "keyword2"]
2. Add a detection block in the parse() method (watch priority order vs. existing keywords)
3. Add an execution handler in executor.py: _execute_my_op(intent, context) -> ExecutionResult
4. Register the handler in NLExecutor._handlers dict
5. Add a test case in tests/test_nl.py covering both parse() and execute_intent()
6. Update README.md: add to the supported operations table

9. Release Notes

v2.2.1 (2025-03-07)

- README fully restructured: human mobility analysis given its own chapter with NL usage, Python API, demo, and metrics reference
- NL keyword mapping fixes: '出行方式构成图' now correctly maps to mobility_modal (was mobility_triplegs); '预测出行方式' now maps to mobility_transport (was unknown)
- Keyword priority hardened: split transport/modal keywords from triplegs keywords to prevent ambiguous matching
- Test versions updated to v2.2.1 across all four test files

v2.2.0 (2025-03-07)

- New: data/mobility/ directory with wuhan_gps_tracks.csv (37,549 GPS points, 5 users, 10 days)
- New: users_meta.csv with user profiles (profession, home area, work area, commute mode)
- New: examples/wuhan_mobility_demo.py — 7-step full pipeline demo generating 5 visualization charts
- New: data/mobility/README.md with data format specification and usage guide
- Fix: read_positionfixes() now uses format='mixed' for robust ISO8601 timestamp parsing
- Fix: radius_of_gyration() column name compatibility (radius_gyration vs radius_of_gyration)
- Visualization: all charts use English labels only (eliminated CJK font dependency)
- Attribution: trackintel algorithm source clearly documented in README and CHANGELOG

v2.1.0 (2025-03-07)

- New module: geoclaw_claude/analysis/mobility/ (core.py + metrics.py + visualization.py)
- New NL operations: mobility_load, mobility_staypoints, mobility_triplegs, mobility_transport, mobility_hierarchy, mobility_locations, mobility_summary, mobility_plot, mobility_heatmap, mobility_modal
- New tests: tests/test_mobility.py with 20 test cases (M01–M20), all passing
- generate_full_hierarchy() as one-call convenience wrapper for the entire trackintel pipeline
- identify_home_work() wrapping trackintel location_identifier() with OSNA and frequency methods

v2.0.0 (2025-03-07)

- Major release: natural language GIS platform, full documentation sync with code
- All test suites verified: memory 37/37, updater 20/20, nl 20/20
- API fully backward compatible with v1.x

v1.3.0 (2025-03-07)

- New module: geoclaw_claude/nl/ (processor.py + executor.py + agent.py)

- New CLI commands: geoclaw-claude ask (single command) and geoclaw-claude chat (multi-turn dialog)
- NL AI mode: Claude claude-3-haiku; Rule mode: 30+ action types via keyword matching
- GeoAgent maintains layer context across turns and auto-flushes to Memory on end()

v1.2.0 (2025-03-07)

- New module: geoclaw_claude/updater.py
- CLI commands: geoclaw-claude check, update, self-check
- self_check() validates 18 modules, 12 dependencies, config, and data directory

v1.1.0 (2025-03-07)

- New module: geoclaw_claude/memory/ (short_term.py + long_term.py + manager.py)
- ShortTermMemory: RLock thread safety, TTL-based eviction, operation log
- LongTermMemory: JSON persistence to ~/.geoclaw_claude/memory/, keyword search, importance scoring
- MemoryManager singleton via get_memory(); CLI memory command group with 8 subcommands
- Bug fix: nearest_neighbor() distance was returning 0 for projected layers
- New function: kde() in spatial_ops.py

v1.0.0 (2025-03)

- First stable release: geoclaw-claude CLI (onboard/config/test/download/skill commands)
- Complete analysis modules: spatial_ops, network, raster_ops, cartography, coord_transform
- Skill system: SkillManager, SkillContext, builtin hospital_coverage skill
- Data: 7 Wuhan GeoJSON demo datasets (hospitals, metro stations, districts, parks, roads, etc.)

10. Attribution & License

10.1 GeoClaw-claude

Field	Value
Project	GeoClaw-claude
Author	UrbanComp Lab
Website	https://urbancomp.net
Repository	https://github.com/whuyao/GeoClaw_Claude
License	MIT License
Copyright	© 2025 UrbanComp Lab

10.2 Third-Party Attributions

⚠️ The mobility analysis module's core algorithms originate from trackintel (ETH Zurich). GeoClaw-claude provides an API layer on top of these algorithms. Original algorithm implementations remain the intellectual property of the trackintel authors.

Package	License	Author	Reference
trackintel	MIT	ETH Zurich · Mobility Information Engineering Lab	Martin et al. (2023). Computers, Environment and Urban Systems, 101. DOI: 10.1016/j.compenvurbsys.2023.101938
osmnx	MIT	Geoff Boeing (2017)	Boeing (2017). Urban Morphology and Design. DOI: 10.1016/j.compenvurbsys.2017.05.004
geopandas	BSD-3	GeoPandas developers	https://geopandas.org
shapely	BSD-3	Sean Gillies et al.	https://shapely.readthedocs.io
networkx	BSD-3	NetworkX developers	https://networkx.org
folium	MIT	Rob Story et al.	https://python-visualization.github.io/folium
rasterio	BSD-3	Mapbox	https://rasterio.readthedocs.io
matplotlib	PSF	Matplotlib developers	https://matplotlib.org